

PROJECT REPORT

Automated Deployment System for React Application

CI/CD Pipeline using GitHub, Jenkins, Docker and AWS EC2

Prepared by: M.Akshaya

GUVI

September 2026

1. Introduction

Modern web applications need to be built, tested, and released quickly and reliably. Manually copying files to a server, installing dependencies, and restarting services every time code changes is slow and error-prone. This project, the Automated Deployment System for React Application, addresses this by building an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline that takes source code from a GitHub repository and delivers it as a running, containerized application on a cloud server without manual intervention.

The pipeline is built around four core technologies working together: Git and GitHub for version control and source hosting, Docker for packaging the React application into a portable container image, Jenkins for orchestrating the build-test-deploy pipeline, and Amazon EC2 as the cloud host that runs the final container and serves the application to end users.

This report documents the objectives, technology choices, architecture, step-by-step implementation, testing approach, issues encountered during development, and the conclusions drawn from building this deployment system.

2. Problem Statement

Traditional deployment of React applications typically involves a developer manually building the application, transferring the build artifacts to a server, configuring the runtime environment, and restarting the web server. This approach has several drawbacks:

- It is time-consuming and repetitive, especially when releases happen frequently.
- It is prone to human error, such as missing configuration steps or deploying the wrong build.
- Environment inconsistencies between the developer's machine, the build server, and production often cause deployment failures.
- There is no repeatable, auditable process for tracking what was deployed and when.
- Scaling manual deployment to multiple servers or environments becomes impractical.

There is a need for an automated system that can consistently build, package, and deploy a React application to a cloud server every time new code is pushed to the source repository, while minimizing manual steps and the associated risk of error.

3. Objectives

- Set up a version-controlled source repository for the React application on GitHub.
- Containerize the React application using Docker so that it runs consistently across environments.
- Configure a Jenkins pipeline to automatically build the Docker image whenever new code is committed.
- Push the built Docker image to Docker Hub for centralized image storage and distribution.
- Provision an AWS EC2 instance to act as the deployment target and host the running application.
- Automate deployment of the Docker container onto the EC2 instance via Jenkins, using SSH.
- Verify that the deployed application is accessible over the public internet and functions correctly.
- Document common issues encountered during setup and their solutions for future reference.

4. Technologies Used

4.1 Version Control

- Git – distributed version control system used to track source code changes.
- GitHub – remote repository hosting used to store and share the project source code.

4.2 Containerization

- Docker – used to package the React application, its dependencies, and runtime into a single portable image.
- Docker Hub – used as the registry to store and distribute the built Docker images.

4.3 Continuous Integration / Continuous Deployment

- Jenkins – automation server used to orchestrate the build, image push, and deployment pipeline.
- Jenkins SSH Agent Plugin – used to allow Jenkins to securely connect to the EC2 instance and run deployment commands.

4.4 Cloud Infrastructure

- Amazon EC2 (Amazon Linux) – cloud virtual machine used to host the deployed application.
- AWS Security Groups – used to control inbound network access to the EC2 instance (SSH on port 22, HTTP on port 80).

4.5 Application

- React – JavaScript library used to build the front-end application that is deployed by this system.

4.6 Client Tools

- Windows PowerShell – used locally to run Git and setup commands during development.

5. Architecture

The system follows a standard CI/CD pipeline architecture consisting of four stages: source, build, registry, and deployment.

```
Developer
|  git push
v
GitHub Repository (devops-build)
|  webhook / poll
v
Jenkins Server
|  1. Checkout source code
|  2. docker build -> react-app image
|  3. docker push -> Docker Hub
|  4. SSH into EC2 instance
v
AWS EC2 Instance
|  docker pull react-app image
|  docker run -p 80:<app-port> react-container
v
End Users (via EC2 Public IP, port 80)
```

In this architecture, GitHub acts as the single source of truth for application code. Jenkins listens for changes, builds a fresh Docker image containing the React application, and pushes that image to Docker Hub so it can be pulled from anywhere. Jenkins then connects to the EC2 instance over SSH and instructs Docker on that instance to pull the latest image and run it as a container, exposing the application on port 80 so it is reachable from the public internet.

6. Implementation Steps

6.1 Local Project Setup

The project folder was created locally and PowerShell was used to navigate into it:

```
cd "D:\guvi\AUTOMATED DEPLOYMENT SYSTEM FOR REACT APPLICATION"
```

Quotation marks were required around the path because the folder name contains spaces; without them, PowerShell would interpret each space-separated word as a separate argument.

6.2 Cloning the Repository

The GUVI GitHub repository containing the application source was cloned into the project folder:

```
git clone https://github.com/sriram-R-krishnan/devops-build.git
```

6.3 Containerizing the Application

A Dockerfile was created for the React application to define how the image should be built, including installing dependencies, building the production bundle, and serving it. The image was then built and tagged:

1. Write a Dockerfile describing the Node/React build stage and the serving stage.
2. Build the Docker image locally with `docker build -t react-app:dev`.
3. Tag and push the image to Docker Hub for later use by the EC2 instance.

6.4 Setting Up the EC2 Instance

- Launched an Amazon Linux EC2 instance and configured its Security Group to allow inbound traffic on port 22 (SSH) and port 80 (HTTP).
- Installed Docker on the EC2 instance and started and enabled the Docker service.
- Added the `ec2-user` to the docker group so Docker commands could be run without `sudo`.

6.5 Configuring the Jenkins Pipeline

- Installed Jenkins and the required plugins, including the SSH Agent plugin.
- Added Docker Hub credentials (`dockerhub-credentials`) and the EC2 SSH key (`ec2-ssh-key`) to the Jenkins credential store.
- Created a Jenkins pipeline job that checks out the GitHub repository, builds the Docker image, pushes it to Docker Hub, and connects to the EC2 instance over SSH to pull and run the updated container.

6.6 Deploying the Container

On the EC2 instance, Jenkins pulls the latest image and (re)starts the application container:

```
docker pull <dockerhub-username>/react-app:dev
docker stop react-container || true
docker rm react-container || true
docker run -d --name react-container -p 80:3000 <dockerhub-username>/react-app:dev
```

7. Testing and Verification

After each deployment, the following checks were performed to confirm the pipeline and the running application were working correctly:

1. Verified the Jenkins build completed successfully with no failed stages in the pipeline console output.
2. Confirmed the new image was visible under the correct repository and tag on Docker Hub.
3. Connected to the EC2 instance and ran `docker ps` to confirm the react-container was up and running.
4. Opened the EC2 instance's public IP address in a browser to confirm the React application loaded correctly.
5. Made a small code change, pushed it to GitHub, and confirmed the pipeline automatically rebuilt and redeployed the updated application.

These checks confirmed that the end-to-end pipeline — from a Git push to a live, updated application — functioned as intended.

8. Problems and Solutions

This section documents the issues encountered while setting up the deployment pipeline and how each one was resolved.

1. Docker command not found on EC2

Problem: Docker commands did not work because Docker was not installed or the Docker service was not running.

Solution:

```
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
```

2. Permission denied while running Docker

Problem: The `ec2-user` account did not have permission to run Docker commands.

Solution:

```
sudo usermod -aG docker ec2-user
```

After running the command, log out and log in again for the group change to take effect.

3. Docker container name already exists

Problem: A container named `react-container` was already running or stopped from a previous deployment, which blocked starting a new one with the same name.

Solution:

```
docker stop react-container
docker rm react-container
```

Then run the new container.

4. React application was not accessible

Problem: The application could not be opened through the EC2 public IP. Possible causes included: port 80 not allowed in the EC2 security group, the Docker container not running, or the application mapped to the wrong port.

Solution:

```
docker ps
```

Check the container status and make sure port 80 is allowed in the EC2 security group.

5. Docker image was not found

Problem: The EC2 instance could not pull the Docker image from Docker Hub.

Solution:

```
docker pull <dockerhub-username>/react-app:dev
```

Check the Docker Hub repository name and image tag carefully to make sure they match exactly.

6. Jenkins deployment failed

Problem: Jenkins could not connect to the EC2 instance through SSH. Possible causes included an incorrect EC2 IP address, an incorrectly configured SSH key, port 22 being blocked, or the Jenkins SSH Agent plugin being missing.

Solution:

- Verify the EC2 public IP address.
- Check the ec2-ssh-key credential in Jenkins.
- Allow SSH port 22 in the EC2 security group.
- Install and configure the SSH Agent plugin.

7. Docker Hub authentication failed

Problem: Jenkins could not push the Docker image to Docker Hub.

Solution:

- Verify the Docker Hub username.
- Use a Docker Hub access token instead of a password.
- Check the dockerhub-credentials credential in Jenkins.
- Confirm that the repository name in the push command matches the Docker Hub repository.

9. Conclusion

The Automated Deployment System for React Application successfully demonstrates a complete CI/CD pipeline that takes source code from GitHub and delivers it as a running, containerized application on AWS EC2 with no manual deployment steps. By combining Git for version control, Docker for consistent packaging, Jenkins for pipeline automation, and EC2 for hosting, the project removes the repetitive and error-prone manual work involved in traditional deployment.

Along the way, several practical issues were encountered — including Docker permission errors, container naming conflicts, network/security group misconfiguration, and Jenkins-to-EC2 connectivity problems — and each was diagnosed and resolved, resulting in a stable and repeatable pipeline. This project provides a solid foundation that can be extended with features such as automated testing before deployment, blue-green or rolling deployments, HTTPS termination, and monitoring/alerting for production readiness.